Actividad en Clase

Objetivo:
Diseñar y desarrollar una aplicación web funcional desde cero, para luego aplicar los principios de calidad de software escribiendo pruebas unitarias, midiendo su efectividad con reportes de cobertura, e investigando cómo implementar una prueba de aceptación que simula la interacción de un usuario real.

Duración: 60- 90 minutos.

Requisitos:

- Conocimientos básicos de Python (u otro lenguaje de programación), HTML y Git.
- Curiosidad para investigar soluciones a problemas nuevos.

---

1. Diseño y Desarrollo Rápido de una Aplicación

Conceptos Aplicados: Desarrollo de software, arquitectura simple (front-end/back-end).

1. **Elegir una idea:** Deben crear una aplicación web simple que no sea una calculadora.
    - **Sugerencias:**
        - Un gestor de "Cosas por Hacer" (To-Do List).
        - Un generador de contraseñas aleatorias.
        - Una aplicación de notas simples.
2. **Requisitos:**
    - **Backend:** Puede ser en **Python o cualquier lenguaje** de tu preferencia (usando un micro-framework como Flask o FastAPI). Debe exponer al menos **3 métodos/endpoints** (ej., para una To-Do list: agregar_tarea, listar_tareas, eliminar_tarea).
    - **Frontend:** Una interfaz de usuario simple en **HTML**. Se permite el uso de **herramientas de IA** (como Gemini, Chat GPT o GitHub Copilot) para generar el código del front-end y el backend.
3. **Desarrollar:** El objetivo es tener una aplicación funcional, aunque sea simple.

2. Pruebas Unitarias y Cobertura de Código

Conceptos Aplicados: Pruebas Unitarias, Cobertura de Código, Estrategia AAA (Arrange, Act, Assert).

1. **Escribir Pruebas Unitarias:**
   - Usando un framework cómo pytest (python), escriban al menos una prueba unitaria para cada uno de los métodos del backend que crearon.
   - Asegúrense de que las pruebas validan el comportamiento esperado de forma aislada. Por ejemplo, si prueban agregar_tarea, deben verificar que la lista interna de tareas realmente contiene la nueva tarea.
2. **Investigar y Medir la Cobertura:**
   - **Investigación:** ¿Cómo se mide la cobertura de código en un proyecto de Python o su lenguaje con pytest o la herramienta escogida?
   - **Implementación:** Instalen la herramienta necesaria y ejecuten las pruebas de forma que se genere un **reporte de cobertura**.
   - **Análisis:** Analicen el reporte. ¿Qué porcentaje de su código está cubierto por las pruebas? ¿Hay líneas o ramas de código que no se están probando?

---

3. Investigación y Creación de una Prueba de Aceptación

Conceptos Aplicados: Pruebas de Aceptación, Pruebas E2E, Automatización de UI.

1. **Investiga el "Cómo":** Si su aplicación funciona cuando un usuario hace clic en los botones. ¿Cómo podemos automatizar eso?
   - **Pregunta de investigación:** "¿Cómo puedo escribir una prueba que abra un navegador, interactúe con mi aplicación en ejecución y verifique el resultado en la página?"
   - **Palabras clave para buscar:** Selenium Python Flask, Playwright pytest, pruebas E2E Python (si usan otro lenguaje deben buscar otras herramientas, aunque la prueba de aceptación puede escribirse en un lenguaje diferente a la aplicación en sí).
2. **Implementar UNA Prueba Sencilla:**
   - Escriban un script de prueba que realice un flujo de usuario simple. Por ejemplo, para la To-Do list:
     1. Inicia la aplicación.
     2. Abre un navegador y navega a la página principal (localhost).
     3. Encuentra el campo de texto para la nueva tarea y el botón de "Agregar".
     4. Escribe "Comprar pan" y hace clic en el botón.
     5. **Verifica (Assert)** que el texto "Comprar pan" ahora aparece en algún lugar de la página.